DUOC UC ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

Especificación de Requisitos de Software

Proyecto Clínica Nutricional NutriVida

Autores Felipe Araya y Antonia Muñoz

DSY1104 Desarrollo Fullstack II

Evaluación Parcial 2 ERS V2

Revisión 2.1 18 de octubre de 2026

Actualización de la propuesta inicial y especificación del frontend React, con criterios de aceptación y trazabilidad de los requisitos.

Ficha del documento

Fecha	Revisión	Autor	Modificación
12/09/2026	1.0	Equipo	Puntos 1 a 2 de la propuesta inicial.
14/09/2026	2.0	Equipo	Versión previa completada para la primera etapa.
18/10/2026	2.1	Felipe Araya y Antonia Muñoz	ERS V2 para EP2. Migración React, ampliación del caso y contraste con los 29 requisitos originales.

Estado de validación: documento actualizado para revisión académica. La aceptación del docente y cualquier firma de conformidad quedan pendientes; no se registra una aprobación que no haya ocurrido.

La numeración R.1 a R.29 conserva la planilla inicial. R.30 a R.42 identifican ampliaciones de la segunda evaluación. “Implementado” describe lo observado en el código; no implica aceptación formal ni ausencia de defectos. “Parcial” identifica diferencias concretas con el requisito original.

Contenido

1 Introducción y alcance

2 Descripción general del producto

3 Requisitos específicos e interfaces

3.2 Requisitos funcionales originales y ampliaciones

3.3 Requisitos no funcionales

3.4 Datos y reglas del negocio

4 Trazabilidad y aceptación

1 Introducción

1.1 Propósito

Esta ERS define los requisitos de NutriVida y los criterios que permiten verificar su implementación en la Evaluación Parcial 2. Está dirigida al equipo de desarrollo, al docente y a quienes revisen el prototipo. Actualiza la propuesta inicial de la clínica, conserva sus identificadores y separa la funcionalidad existente de los puntos pendientes.

1.2 Ámbito del sistema

NutriVida digitaliza la consulta de servicios, el agendamiento y el seguimiento nutricional de una clínica ficticia. Los pacientes pueden registrarse, editar su perfil, reservar y cancelar citas, consultar su progreso y leer el plan asignado. El nutricionista consulta su agenda y asigna planes; el administrador mantiene usuarios, servicios, categorías, citas y reportes.

La segunda etapa incorpora carrito, checkout y resultados de pago simulados para adaptar el caso de tienda de la evaluación. La compra guarda una orden y un comprobante académico; no reserva una hora automáticamente. La reserva de atención se realiza en un flujo separado.

El alcance implementado es un frontend React con persistencia local. No procesa cobros bancarios reales, no emite documentos tributarios, no realiza diagnósticos automáticos ni administra historiales médicos externos. El backend Java, la base de datos central y la autenticación de servidor son trabajo futuro. La mención de integración backend del anexo debe confirmarse con el docente.

1.3 Definiciones acrónimos y abreviaturas

Término	Definición
ERS	Especificación de Requisitos de Software.
SPA	Aplicación de una sola página con rutas internas.
CRUD	Crear, consultar, actualizar y eliminar registros.
Props y estado	Datos recibidos y valores que cambian durante la interacción.
DOM	Modelo de objetos que representa el documento en el navegador.
localStorage	Almacenamiento por origen que persiste entre recargas.
RUN	Rol Único Nacional, identificador utilizado en los formularios del prototipo.
Mock y spy	Sustitución o interceptación controlada de una dependencia.
Cobertura	Código recorrido por las pruebas; no equivale a casos aprobados.

Término	Definición
Comprobante académico	Resumen de una compra simulada, sin validez tributaria.

1.4 Referencias

ERS completado de la Evaluación Parcial 1, Anexo 4, archivo terminado en (2), revisión 2.0 del 14/09/2026.

Plantilla original de ERS, Anexo 4, archivo terminado en (3), estructura de referencia basada en IEEE 830.

Planilla de Requerimientos, Evaluación Parcial 1, Anexo 2, hoja Requerimiento Inicial, R.1 a R.29.

DSY1104 Evaluación Parcial 2, Anexo 1 Instrucciones, páginas 1 a 12, y pauta EE EP Estudiante, páginas 1 a 9.

Repositorio del proyecto: https://github.com/AMunozPSo/Clinica-Nutricional-NutriVida. Base de código revisada: commit 18ba6dc.

Informe de testing y evidencias locales de la verificación del 18/10/2026, 20:42, conservados en nutrivida-react/docs. Las últimas cargas de ejemplos son posteriores a esa medición.

1.5 Visión general del documento

La sección 2 presenta usuarios, restricciones y dependencias. La sección 3 especifica interfaces, requisitos funcionales y no funcionales, reglas y datos. La sección 4 relaciona las exigencias de la pauta con evidencias de implementación y pruebas, e identifica los puntos que requieren corrección o aceptación.

2 Descripción general

2.1 Perspectiva del producto

La solución evoluciona desde HTML, CSS y JavaScript a una SPA desarrollada con React, Vite, React Router y React Bootstrap. App define rutas; Navbar, Footer y TarjetaServicio reutilizan presentación; los formularios manejan su estado con hooks. Los módulos storage, admin, booking, account, commerce y clinical encapsulan datos y validaciones sobre localStorage.

La arquitectura cliente servidor descrita en la versión anterior sigue siendo una evolución futura. En esta etapa los datos no se comparten entre dispositivos ni entre orígenes distintos del navegador. Las rutas privadas y los roles se simulan en el cliente.

2.2 Funciones del producto

La interfaz pública presenta Inicio, Nosotros, Contacto, Blog, categorías, catálogo, ofertas y detalle de servicios. El usuario puede buscar y filtrar, iniciar sesión, gestionar su perfil, consultar y cancelar citas, comprar servicios mediante un pago de prueba y consultar sus compras. La atención incluye progreso personal y plan vigente.

El panel administrativo incorpora resumen, usuarios, servicios, categorías, citas, órdenes, reportes y servicios incompletos. La agenda del nutricionista muestra sus citas asignadas y permite publicar planes para pacientes autorizados. Los datos de ejemplo se cargan una sola vez por versión y después pueden editarse o eliminarse.

2.3 Características de los usuarios

Actor	Características y tareas
Visitante	Conocimientos básicos de navegador; consulta contenido, catálogo y ofertas, se registra y puede probar la compra como invitado.
Paciente	Completa formularios, consulta su información y gestiona sus propias citas, compras, progreso y plan.
Nutricionista	Manejo básico de herramientas digitales; consulta su agenda y asigna planes a sus pacientes. No registra mediciones clínicas de terceros en esta versión.
Administrador	Manejo de paneles y roles; mantiene el catálogo y cuentas, revisa citas, historial, órdenes y reportes. Las cuentas principales gestionan privilegios elevados.
Equipo de desarrollo	Mantiene código, pruebas y documentos; reproduce la aplicación y explica su diseño y validación en la defensa.

2.4 Restricciones

Para EP2 se requiere React, gestión de props y estado, navegación y formularios, diseño adaptable con Bootstrap y pruebas con Jasmine y Karma. Se retira la prohibición de frameworks de la Evaluación Parcial 1, porque contradice la tecnología solicitada en esta etapa.

La persistencia simulada se implementa en archivos JavaScript que leen y escriben localStorage. Se deben mantener los formatos heredados utilizados por el HTML sin borrar datos del usuario. Las credenciales y permisos de demostración no deben emplearse con información clínica real.

No se presupone que el docente haya aceptado cambios respecto de la planilla: las diferencias de RUN, mapa, reprogramación, desactivación y cupos quedan identificadas en sus requisitos. No se sustituyen silenciosamente por funciones distintas.

2.5 Suposiciones y dependencias

Se requiere navegador moderno con JavaScript y Web Storage habilitados. Para instalar y verificar el proyecto se utiliza Node.js 24, dependencias del archivo de bloqueo y Chrome o Edge para Karma. La interfaz conserva sus datos mientras se use el mismo origen; limpiar el almacenamiento elimina los registros locales.

OpenStreetMap y las fuentes externas requieren conexión. El mapa está embebido por iframe y no utiliza actualmente Leaflet. La reserva aplica fecha y hora de America/Santiago; la prueba de aceptación debe usar un día hábil futuro y un horario libre.

2.6 Requisitos futuros

Integrar una API Java y Spring Boot con una base relacional, sustituir la identidad simulada por autenticación de servidor y definir permisos verificables. Incorporar respaldo central, trazabilidad de operaciones, registro profesional de mediciones, notificaciones y, si se autoriza, pasarela de pagos real.

Antes de declarar aceptación completa de la propuesta inicial deben resolverse los requisitos parciales de la sección 4. La reprogramación, la alerta de cupos y la desactivación reversible no se declaran implementadas por aparecer como objetivos futuros.

3 Requisitos específicos

3.1 Requisitos comunes de las interfaces

3.1.1 Interfaces de usuario

Las páginas públicas utilizan navegación superior adaptable y footer compartido. Los formularios tienen etiquetas, mensajes contextuales y botones de acción. Admin utiliza menú lateral, tablas responsivas y confirmaciones para acciones sensibles. El saludo abre el perfil; el menú refleja el rol de la sesión.

Entradas: credenciales, datos de registro y perfil, búsqueda, categoría, servicio, fecha, hora, motivo, cantidades y datos del checkout. Salidas: tarjetas, listados, agenda, plan, progreso, alertas y comprobantes. Cuando no existen resultados se informa un estado vacío y se conserva la navegación.

3.1.2 Interfaces de hardware

La aplicación debe poder utilizarse con teclado, ratón y pantalla táctil en móvil, tableta y escritorio. No requiere periféricos clínicos ni conexión con equipos de medición. Las resoluciones de aceptación son 360, 768 y 1280 píxeles, sujetas a revisión visual pendiente.

3.1.3 Interfaces de software

React renderiza componentes; React Router resuelve navegación; Bootstrap y React Bootstrap aportan grillas, formularios, tablas y navegación. Web Storage guarda JSON. OpenStreetMap proporciona el mapa embebido. Vitest y Testing Library complementan la ejecución obligatoria de Jasmine y Karma.

3.1.4 Interfaces de comunicación

El servidor de desarrollo y el build sirven archivos mediante HTTP o HTTPS. Esta versión no intercambia operaciones de negocio con una API de servidor: los módulos JavaScript actualizan datos locales y notifican a React mediante eventos. La futura API REST intercambiará JSON y deberá definir autenticación y contratos de errores.

3.2 Requisitos funcionales

Los textos de alcance de R.1 a R.29 proceden de la planilla inicial. Cada requisito incorpora aceptación, estado observado y ubicación de implementación. Los criterios pendientes permanecen como exigencias de la propuesta, sin asumir que estén aprobados o cancelados.

R.1 Vista de catálogo de servicios

Actor: Paciente. Mostrar el catálogo completo de servicios (consultas, planes especializados, evaluaciones y talleres) en formato de tarjetas responsivas, indicando nombre, precio, duración, modalidad y una breve descripción.

Aceptación: Cada tarjeta debe mostrar nombre, precio, duración, modalidad y descripción, con enlace al detalle.

Estado: Parcial. Las tarjetas muestran nombre, precio y descripción; faltan duración y modalidad visibles. Evidencia: TarjetaServicio.jsx.

R.2 Filtro de catálogo por categoría

Actor: Paciente. Permitir filtrar el catálogo por tipo de servicio (Consulta, Plan especializado, Evaluación, Taller grupal) y por modalidad (Presencial/Online).

Aceptación: La categoría y la modalidad deben poder combinarse con la búsqueda, incluyendo un estado sin resultados.

Estado: Parcial. Existen filtros por categoría y texto; falta el selector Presencial/Online. Evidencia: Catalogo.jsx.

R.3 Buscador de catálogo

Actor: Paciente. Incluir un campo de búsqueda por texto que filtre en tiempo real los servicios del catálogo por nombre o palabra clave.

Aceptación: Al escribir un nombre o palabra de la descripción se actualizan las tarjetas sin recargar la página.

Estado: Implementado. Búsqueda mediante estado React y filtrado del catálogo. Evidencia: Catalogo.jsx; Shop.test.jsx.

R.4 Selección de servicio hacia agendamiento

Actor: Paciente. Al presionar 'Agendar' sobre un servicio del catálogo, precargar automáticamente su nombre, precio y duración en el formulario de solicitud de cita.

Aceptación: Agendar desde el detalle debe conservar el servicio y mostrar su nombre, precio y duración antes de confirmar.

Estado: Parcial. El servicio y profesional se precargan; el resumen previo no muestra precio ni duración. Evidencia: DetalleServicio.jsx; AgendarCita.jsx.

R.5 Formulario de solicitud de cita

Actor: Paciente. Formulario de solicitud de cita con nutricionista preferido, motivo de consulta, fecha/hora y servicio seleccionado, con validación de campos obligatorios antes del envío.

Aceptación: Validar servicio, profesional, motivo según obligatoriedad acordada, día hábil y horario disponible antes de guardar.

Estado: Parcial. El profesional depende del servicio y el motivo es opcional; falta un selector independiente de profesional si se exige como preferencia. Evidencia: AgendarCita.jsx; booking.js.

R.6 Vista 'Mis citas'

Actor: Paciente. Listado de las citas del paciente con su estado (pendiente, confirmada, cancelada) y acciones para cancelar o solicitar reprogramación.

Aceptación: Mostrar solo citas propias, permitir cancelar con confirmación y solicitar cambio de fecha u hora.

Estado: Parcial. Consulta, filtro y cancelación implementados; la reprogramación directa no está implementada. Evidencia: MisCitas.jsx; account.js.

R.7 Formulario de inicio de sesión y registro

Actor: Paciente. Vistas de login y registro con validaciones en tiempo real (formato de correo, largo mínimo de contraseña) y mensajes de error contextuales.

Aceptación: Validar correo, contraseña y campos del registro, informar errores contextuales y evitar cuentas duplicadas.

Estado: Parcial. Login y registro funcionan; las validaciones se ejecutan al enviar, no todas en tiempo real como indica la propuesta inicial. Evidencia: Login.jsx; Registro.jsx; registration.js.

R.8 Vista de historial de progreso

Actor: Paciente. Panel con un gráfico simple de peso/medidas en el tiempo para que el paciente visualice su evolución entre sesiones.

Aceptación: Guardar fecha y peso, mostrar evolución cronológica y permitir eliminar registros propios; rechazar datos inválidos.

Estado: Implementado. Gráfico de barras y tabla de peso; no se han incorporado otras mediciones clínicas. Evidencia: Progress.jsx; clinical.js.

R.9 Vista de plan alimenticio vigente

Actor: Paciente. Sección de solo lectura donde el paciente consulta el plan alimenticio activo asignado por su nutricionista.

Aceptación: El paciente autenticado debe consultar únicamente el plan vigente asociado a su correo.

Estado: Implementado. Consulta personal de solo lectura; el profesional asigna el plan desde su agenda. Evidencia: Plan.jsx; Doctor.jsx; clinical.js.

R.10 Menú de navegación responsivo

Actor: Paciente, Administrador. Barra de navegación que se colapsa en un menú tipo 'hamburguesa' en móvil (≥360px) y se expande en tablet/escritorio.

Aceptación: Comprobar menú y navegación a 360, 768 y 1280 píxeles, incluyendo apertura y cierre del menú móvil.

Estado: Pendiente de validación visual. Navbar de React Bootstrap implementada; falta registrar una revisión visual en los tres tamaños. Evidencia: Navbar.jsx; index.css.

R.11 Vista de ubicación con mapa

Actor: Paciente. Sección con un mapa interactivo embebido (Leaflet) que muestra la ubicación de la clínica.

Aceptación: Mostrar la ubicación de la clínica en un mapa navegable y verificar su carga con conexión a internet.

Estado: Parcial. Se utiliza un iframe interactivo de OpenStreetMap; no se integra Leaflet como pedía el requisito original. Evidencia: Contacto.jsx.

R.12 Panel de administración de usuarios

Actor: Administrador. Vista tipo CRUD (crear, editar, desactivar, asignar rol) visible únicamente para el rol Administrador, con confirmación antes de desactivar un usuario.

Aceptación: El administrador debe crear, editar, desactivar y gestionar roles con confirmación y protección de cuentas principales.

Estado: Parcial. Creación, edición, eliminación y roles implementados; falta desactivación reversible sin eliminar la cuenta. Evidencia: Users.jsx; admin.js.

R.16 Vista "Nosotros"

Actor: Paciente. Página institucional que presenta la historia de la clínica, su misión y al equipo de nutricionistas.

Aceptación: Presentar información institucional, misión y equipo profesional con fotografías y navegación accesible.

Estado: Implementado. Vista institucional y carrusel implementados. Evidencia: Nosotros.jsx.

R.17 Vista de blogs / noticias

Actor: Paciente. Listado de noticias o datos curiosos de la clínica, mostrados en tarjetas con imagen, título y descripción corta.

Aceptación: Listar al menos dos artículos con imagen, título, resumen y enlace al contenido completo.

Estado: Implementado. Dos artículos educativos con navegación desde el listado. Evidencia: Blog.jsx; Shop.test.jsx.

R.18 Detalle de blog

Actor: Paciente. Vista de detalle para cada noticia del blog (mínimo 2), con imagen, título y descripción larga.

Aceptación: Abrir cada artículo mediante su ruta y mostrar título, imagen y texto ampliado; gestionar rutas no existentes.

Estado: Implementado. Detalle de los dos artículos implementado en React. Evidencia: Blog.jsx; App.jsx.

R.19 Formulario de contacto

Actor: Paciente. Formulario para que el paciente envíe un mensaje interno (nombre, correo, comentario), con validación de campos obligatorios antes de enviar.

Aceptación: Validar nombre, correo y comentario, guardar el mensaje y mostrar confirmación solo después del guardado.

Estado: Implementado. Persistencia local del mensaje y retroalimentación; no hay envío de correo real. Evidencia: Contacto.jsx; Contacto.test.jsx.

R.20 Selector dependiente de Región y Comuna

Actor: Paciente. Al cambiar la región seleccionada en el formulario de registro, actualizar dinámicamente el listado de comunas disponibles usando un arreglo de datos en JavaScript.

Aceptación: Al cambiar región se actualizan comunas y se descarta cualquier comuna incompatible.

Estado: Implementado. Datos de cuatro regiones de ejemplo en JavaScript; no representa el catálogo completo de Chile. Evidencia: registration.js; Registro.jsx; Perfil.jsx.

R.21 Vista de detalle de servicio

Actor: Paciente. Página individual de cada servicio del catálogo con descripción extendida, profesional asignado y servicios relacionados, accesible al hacer clic desde la tarjeta del catálogo.

Aceptación: El detalle debe mostrar descripción ampliada, profesional y servicios relacionados de la misma categoría.

Estado: Parcial. Descripción, profesional, precio y acciones disponibles; falta el bloque de servicios relacionados. Evidencia: DetalleServicio.jsx.

R.22 Vista de agenda del nutricionista

Actor: Nutricionista. Listado de citas asignadas al nutricionista, mostrando paciente, fecha/hora y motivo de consulta.

Aceptación: El nutricionista debe ver solo las citas asignadas a su cuenta, con paciente, fecha, hora y motivo.

Estado: Implementado. Se corrige el actor de la planilla de Paciente a Nutricionista; el administrador tiene vista general. Evidencia: Doctor.jsx; clinical.js.

R.24 Almacenamiento local de citas/servicios

Actor: Paciente. Guardar temporalmente los datos del agendamiento o servicios seleccionados utilizando localStorage en el navegador para no perder el progreso si se recarga la página.

Aceptación: Después de guardar y recargar, el catálogo y las citas deben conservarse en el mismo origen del navegador.

Estado: Implementado. Se leen los almacenes actuales y los formatos heredados sin duplicar citas. Evidencia: storage.js; admin.js; booking.js.

R.25 Validación estricta de RUN

Actor: Paciente, Administrador. Validar mediante JavaScript que el RUN ingresado en los formularios de registro y creación de usuarios no contenga puntos ni guion (ej: 19011022K), validando que su longitud sea entre 7 y 9 caracteres.

Aceptación: Aceptar RUN de 7 a 9 caracteres sin puntos ni guion y rechazar los formatos contrarios, conforme a la planilla original.

Estado: Parcial. El registro actual exige guion y 7 u 8 dígitos más verificador; Admin no aplica la misma validación. Debe corregirse o acordarse el cambio. Evidencia: registration.js; admin.js.

R.26 Alerta de cupos limitados (Simulación de Stock)

Actor: Paciente. Mostrar un mensaje dinámico de alerta en los talleres grupales o consultas muy demandadas cuando la capacidad simulada sea baja, adaptando la lógica de "stock crítico" mediante JavaScript.

Aceptación: Definir capacidad y umbral, y mostrar alerta pública cuando los cupos simulados del taller o consulta sean bajos.

Estado: Pendiente de implementación. La agenda evita solapamientos y Admin detecta servicios incompletos, pero no existe una alerta pública de cupos simulados. Evidencia: TarjetaServicio.jsx; Business.jsx.

R.27 Persistencia de sesión simulada

Actor: Todos. Utilizar variables en localStorage para simular el inicio de sesión y renderizar condicionalmente el menú de navegación (ej. mostrar el panel de Administrador o la vista "Mis citas" del paciente).

Aceptación: Conservar la sesión al recargar, adaptar el menú al rol y eliminarla al cerrar sesión.

Estado: Implementado. Autorización simulada en cliente; no equivale a seguridad de un servidor. Evidencia: storage.js; useSession.js; Navbar.jsx.

R.28 Carrusel interactivo de la clínica

Actor: Paciente. Implementar un carrusel de imágenes estático en la página principal (HOME) o "Nosotros" utilizando únicamente HTML, CSS y eventos de JavaScript para navegar entre fotos de la clínica.

Aceptación: Navegar entre fotografías mediante controles del carrusel y mantener la adaptación a pantalla.

Estado: Implementado con actualización de etapa. El carrusel migra a React Bootstrap en la Evaluación 2; se sustituye la restricción de JavaScript puro de la primera etapa. Evidencia: Nosotros.jsx.

R.29 Retroalimentación dinámica (Toast/Alertas)

Actor: Todos. Desplegar sugerencias y mensajes de éxito o error personalizados mediante JavaScript al enviar formularios (contacto, registro, agendamiento), mejorando la usabilidad sin recargar la página.

Aceptación: Mostrar mensajes contextuales de éxito y error en contacto, registro y reserva sin recargar el sitio.

Estado: Implementado. Alert y validación de formularios; los errores de persistencia probados no anuncian éxito. Evidencia: Contacto.jsx; Registro.jsx; AgendarCita.jsx.

Ampliaciones de la Evaluación Parcial 2

R.30 Perfil personal editable

Actor: Paciente, Nutricionista, Administrador. El saludo del menú abre el perfil de la sesión para editar nombre, teléfono, dirección, región y comuna.

Aceptación: Guardar y recargar conserva los cambios; actualizar el nombre cambia el saludo sin modificar rol ni credenciales.

Estado: Implementado. Evidencia: Perfil.jsx; account.js; Account.test.jsx.

R.31 Administración del catálogo

Actor: Administrador. Crear, consultar, editar y eliminar servicios con precio, categoría, profesional, modalidad, duración, imagen y descuento.

Aceptación: Los cambios se reflejan en catálogo y detalles; validar campos y confirmar eliminaciones.

Estado: Implementado. Evidencia: Services.jsx; admin.js; Admin.test.jsx.

R.32 Administración y vista de categorías

Actor: Administrador, Visitante. Gestionar categorías y publicar enlaces al catálogo filtrado.

Aceptación: Impedir eliminar una categoría con servicios asociados y rechazar nombres duplicados.

Estado: Implementado. Evidencia: Business.jsx; Categorias.jsx; commerce.js.

R.33 Ofertas y datos de demostración

Actor: Visitante, Administrador. Publicar servicios con descuento y cargar ejemplos iniciales en las ocho categorías del menú.

Aceptación: La oferta muestra precio original y rebajado; la carga de ejemplos no duplica servicios ni sobrescribe datos existentes y se ejecuta una sola vez por versión.

Estado: Implementado. Evidencia: Catalogo.jsx; demoCatalog.js; commerce.js.

R.34 Carrito por cuenta e invitado

Actor: Visitante, Paciente. Añadir servicios, cambiar cantidades y eliminar ítems, separando el carrito de compras de las reservas.

Aceptación: Aceptar cantidades entre 1 y 10; recalcular totales y adoptar el carrito invitado al iniciar sesión.

Estado: Implementado. Evidencia: Cart.jsx; commerce.js; Shop.test.jsx.

R.35 Checkout con datos de contacto

Actor: Visitante, Paciente. Recoger nombre, correo, teléfono, dirección, región, comuna y modalidad de atención con datos de perfil precargados.

Aceptación: Rechazar datos incompletos y permitir corregir los datos antes de confirmar el pago de prueba.

Estado: Implementado. Evidencia: Checkout.jsx; commerce.js.

R.36 Pago simulado y comprobante

Actor: Visitante, Paciente. Demostrar compra aprobada o rechazada y mostrar un resumen académico con folio, cantidades y precios guardados.

Aceptación: Aprobar guarda orden y vacía carrito; rechazar conserva carrito; repetir una solicitud no duplica la orden.

Estado: Implementado. Evidencia: OrderResult.jsx; commerce.js; Shop.test.jsx.

R.37 Consulta de órdenes e historial

Actor: Paciente, Administrador. Permitir consultar compras propias y, al administrador, órdenes generales e historial por usuario.

Aceptación: Restringir el comprobante al propietario, administrador o última compra del invitado identificado en esa sesión.

Estado: Implementado. Evidencia: Orders.jsx; Users.jsx; Business.jsx.

R.38 Reportes y servicios que requieren revisión

Actor: Administrador. Mostrar resumen del negocio, estados de citas, ingresos de compras simuladas e ingresos de citas completadas por separado.

Aceptación: No sumar una compra y su atención como un mismo ingreso; identificar servicios incompletos para su revisión.

Estado: Implementado. Evidencia: Overview.jsx; Reports.jsx; Business.jsx.

R.39 Asignación del plan alimenticio

Actor: Nutricionista, Administrador. Crear o actualizar el plan de un paciente autorizado desde la agenda del profesional.

Aceptación: El nutricionista solo asigna planes a pacientes de su agenda; el paciente únicamente consulta su propio plan.

Estado: Implementado. Evidencia: Doctor.jsx; Plan.jsx; clinical.js.

3.3 Requisitos no funcionales

R.13 Responsividad

Todas las vistas deben renderizar correctamente en 360px (móvil), 768px (tablet) y 1280px (escritorio), sin scroll horizontal ni elementos cortados.

Aceptación: Revisar todas las vistas a 360, 768 y 1280 píxeles, sin recortes ni desplazamiento horizontal del documento.

Estado: Pendiente de validación visual. Bootstrap, tablas responsivas y CSS adaptativo implementados; no existe acta de revisión visual completa. Evidencia: index.css; admin.css.

R.14 Consistencia visual

Uso de una paleta de colores y tipografía únicas en toda la app mediante componentes CSS reutilizables, evitando animaciones o transiciones complejas.

Aceptación: Mantener tipografía Poppins, componentes compartidos y la paleta institucional en páginas públicas y administrativas.

Estado: Implementado. Navbar, Footer, tarjetas y estilos compartidos; revisión estética final incluida en la aceptación manual. Evidencia: index.css; admin.css; components/.

R.15 Rendimiento de interfaz

Las vistas principales (catálogo, mis citas, historial) deben cargar en menos de 2 segundos en condiciones normales de red, evitando renders bloqueantes.

Aceptación: Medir catálogo, citas e historial en un entorno definido y comprobar cargas inferiores a 2 segundos.

Estado: Pendiente de medición. No se dispone de medición de carga reproducible; el build correcto no demuestra ese tiempo. Evidencia: Build Vite; medición pendiente.

R.23 Control de versiones y documentación de cambios

Todo el código debe versionarse en un repositorio Git remoto (GitHub) con commits descriptivos, permitiendo trazabilidad de los cambios de cada integrante.

Aceptación: Conservar el código y la documentación en un repositorio GitHub público con commits descriptivos.

Estado: Implementado. Repositorio público actualizado; Git identifica la cuenta configurada para cada commit, sin atribuir trabajo no demostrado. Evidencia: GitHub y registro Git.

R.40 Pruebas de componentes con Jasmine y Karma

Ejecutar al menos diez casos que comprueben renderizado, props, estado, eventos y errores en componentes React.

Aceptación: 94 ejecuciones aprobadas en la verificación documentada: 74 casos compartidos y 20 casos nativos adicionales; no representan 168 casos distintos.

Estado: Implementado. Evidencia: karma.conf.cjs; tests/jasmine/; scripts/run-jasmine.mjs.

R.41 Proceso y evidencia de testing

Configurar datos aislados, mocks o spies, restauración del entorno, cobertura y análisis de resultados reproducibles.

Aceptación: El informe identifica herramientas, alcance, métricas, errores simulados y limitaciones. npm run verify termina sin errores y conserva evidencias.

Estado: Implementado. Evidencia: scripts/verify.mjs; docs/evidencias/; Informe_Testing.md.

R.42 Entrega y reproducibilidad

Entregar código React, instrucciones, GitHub público, ZIP, ERS actualizado e informe de cobertura.

Aceptación: Instalar con npm ci, ejecutar npm run dev y verificar con Node.js 24 y Chrome o Edge; revisar que los documentos coincidan con el alcance declarado.

Estado: Documentación actualizada. Evidencia: README.md; package-lock.json; docs/.

3.3.1 Requisitos de rendimiento

R.15 conserva el objetivo de carga inferior a 2 segundos. Para verificarlo deben registrarse dispositivo, navegador, conexión, build y condiciones de caché en catálogo, citas e historial. Se propone realizar cinco cargas por vista y conservar tiempos y capturas; la medición todavía no se ha ejecutado. No se establece una capacidad multiusuario de servidor porque no hay backend compartido.

3.3.2 Seguridad

La aplicación debe impedir desde sus flujos habituales que un paciente abra Admin, consulte órdenes ajenas o asigne planes. Las cuentas principales se protegen frente a borrado y cambios de rol desde la interfaz. Deben confirmarse las eliminaciones y cambios sensibles. Las pruebas comprueban estos límites del prototipo.

localStorage puede ser modificado por quien controla el navegador y las contraseñas se conservan sin protección de servidor. No se declara cifrado, autenticación robusta ni autorización de producción. Para una versión real se requiere proteger credenciales y datos en backend y revisar el tratamiento de información clínica.

3.3.3 Fiabilidad

Una operación fallida no debe presentarse como exitosa. Las pruebas simulan errores de almacenamiento en reserva y progreso; comprueban que no se anuncie un guardado correcto. Las órdenes conservan precios históricos y usan un identificador para impedir duplicación de una misma solicitud. La actualización de almacenamiento no es una transacción de base de datos ni ofrece garantías entre dispositivos.

3.3.4 Disponibilidad

El prototipo debe poder abrirse mientras el servidor local o el alojamiento de archivos esté activo. No se fija un porcentaje de disponibilidad sin infraestructura ni monitorización. El mapa puede no cargar si OpenStreetMap o la conexión fallan; los demás flujos locales deben conservar su navegación. El equipo debe iniciar el servidor y comprobar acceso antes de la demostración.

3.3.5 Mantenibilidad

Los componentes de presentación, páginas, hooks y reglas del dominio deben mantenerse separados, con nombres comprensibles y formato uniforme. Cada modificación relevante debe pasar pruebas, ESLint y build. El equipo mantiene código y documentos; el administrador cambia servicios y categorías desde el panel. La documentación se actualiza cuando cambia el alcance o los criterios de aceptación.

3.3.6 Portabilidad

El código frontend se instala desde el repositorio y archivo de bloqueo con Node.js 24 y se compila a archivos estáticos. Debe verificarse en navegadores modernos de escritorio y móvil; Safari y Firefox no se declaran probados. Las rutas SPA requieren fallback hacia index.html al desplegar. La información local no se migra automáticamente entre navegadores, puertos o equipos.

3.4 Otros requisitos

3.4.1 Datos y persistencia

Conjunto de datos	Contenido y reglas
Usuarios y sesión	nutrivida_usuarios y usuarios; nutrivida_sesion y adminLogueado. Correo como vínculo de cuenta; información por origen del navegador.
Servicios y categorías	serviciosNutriVida y nutrivida_servicios; nutrivida_categorias. La categoría enlaza servicios y filtro público.
Citas	nutrivida_citas y nutrivida_carrito heredado. Correo del paciente, servicio, profesional, fecha, hora, duración, motivo y estado; unión sin duplicados.
Carrito y órdenes	nutrivida_compra por cuenta o invitado; nutrivida_ordenes. Cantidades, precios históricos, contacto, resultado y folio.
Atención y contacto	nutrivida_planes, nutrivida_progreso y nutrivida_mensajes. Plan vigente, fecha y peso personal, mensajes internos.
Ejemplos	Marcadores de inicialización v1 y v2; se agregan datos una sola vez sin sobrescribir registros existentes ni recrear elementos eliminados.

3.4.2 Reglas de negocio

Las reservas se ofrecen de lunes a viernes entre 09:00 y 19:00, con inicios cada 30 minutos y validación de duración y disponibilidad del profesional. No se acepta una hora pasada u ocupada.

Las citas pendientes o confirmadas pueden cancelarse con confirmación; una cita cancelada libera el horario. Las completadas o canceladas no se cancelan otra vez. La reprogramación directa está pendiente.

Una compra aprobada guarda precios y cantidades y vacía el carrito. Una compra rechazada conserva el carrito. Comprar no agenda una consulta. Los descuentos se reflejan en la compra; la reserva utiliza el precio de servicio guardado por booking.js y no garantiza aplicar esa promoción.

Un paciente consulta sus propios datos. El nutricionista ve su agenda y asigna planes a sus pacientes. El administrador revisa datos generales, y solo las cuentas principales otorgan o revocan privilegios de administrador.

El registro admite cuatro regiones y comunas de ejemplo. El rol médico se simula mediante correo de dominio clínico; no constituye validación real de credenciales profesionales.

3.4.3 Documentación y entrega

Entregar enlace GitHub público, proyecto comprimido, este ERS V2 y documento de cobertura de testing. El README debe indicar instalación, ejecución, cuentas de demostración y verificación. Los originales de la primera evaluación se conservan como referencia y no sustituyen la aplicación React ubicada en nutrivida-react.

4 Trazabilidad y aceptación

4.1 Correspondencia con la pauta

Exigencia	Requisitos y evidencia
Framework y estructura	R.1 a R.12, R.16 a R.22, R.27 a R.39. App.jsx, páginas y módulos del dominio.
Props estado y Bootstrap	R.3, R.10, R.13 y R.14. TarjetaServicio, formularios, Navbar y estilos. Validación visual completa pendiente.
Diez pruebas Jasmine y Karma	R.40. Suite de componentes, casos nativos J01 a J20 y configuración Karma.
Entorno mocks y cobertura	R.41. Scripts, aislamiento de datos, spies, logs y análisis de cobertura en el informe.
Entrega documental y GitHub	R.23 y R.42. Repositorio público, ERS, informe, README y ZIP.
Defensa individual	Explicar arquitectura, props, estado, responsividad y pruebas. Guía de 15 minutos más 5 de preguntas.

4.2 Evidencias de pruebas

Verificación	Resultado documentado
Vitest y Testing Library	74 pruebas aprobadas en 10 archivos.
Jasmine y Karma	94 ejecuciones aprobadas en Chrome Headless: 74 compartidas y 20 adicionales.
Cobertura de líneas	Vitest 90.75 %; Jasmine 90.87 %.
Cobertura de ramas	Vitest 80.58 %; Jasmine 80.69 %.
ESLint y build	Sin errores en la verificación registrada.

Estos resultados corresponden a la ejecución del 18/10/2026 a las 20:42 y a la versión descrita en el informe. Los ejemplos de catálogo añadidos después se comprobaron mediante verificaciones específicas de inicialización, conservación de datos, duplicados, imágenes, ESLint y build. No se atribuye a esos cambios una nueva medición de cobertura completa.

Una suite aprobada no demuestra que se haya cumplido cada requisito de la planilla original: algunos criterios no estaban incluidos en los casos existentes. Por ese motivo los estados de la sección 3 se contrastan también con el código y se conservan los pendientes.

4.3 Diferencias que requieren cierre

Requisitos	Acción para su aceptación completa
R.1 y R.4	Mostrar duración y modalidad en tarjetas, y precio y duración en el resumen previo a reservar.
R.2	Añadir filtro por modalidad combinado con categoría y búsqueda.
R.5 y R.7	Acordar selección de profesional y obligatoriedad del motivo; completar validación en tiempo real.
R.6 y R.12	Implementar reprogramación y desactivación reversible de cuentas.
R.11 y R.21	Integrar Leaflet o aprobar el iframe; añadir servicios relacionados al detalle.
R.25 y R.26	Unificar el RUN sin puntos ni guion según planilla y mostrar alerta de cupos simulados.
R.10 R.13 y R.15	Registrar revisión visual en tres resoluciones y medir tiempos de carga.
Alcance de backend	Confirmar con el docente el alcance de la mención de integración backend del anexo; no existe backend real en esta versión.

4.4 Criterio de cierre de la entrega

La documentación queda completada y el frontend es demostrable en el alcance descrito. La aceptación total de la propuesta inicial requiere cerrar o aprobar expresamente las diferencias anteriores y registrar las comprobaciones pendientes. La revisión del documento no reemplaza la demostración funcional ni garantiza una calificación.

En la defensa, cada integrante debe explicar cómo se construyen componentes, cómo se usan props y estado, cómo se adapta la interfaz con Bootstrap y cómo se ejecutan pruebas Jasmine y Karma con mocks y análisis de cobertura. La entrega por encargo pondera 40 % y la presentación individual 60 %, según la pauta suministrada.